



JETBRAINS

Stem Agent: Self-Specializing AI Agent Through Guided Evolutionary Configuration Search

Andrea Gennari

Contents

1	Goal and Scope	2
2	Benchmark and Splits	2
3	Stem-Agent Mechanism	2
4	Current Evidence	3
5	What Surprised Me	4
6	Limitations and Failure Modes	4
7	What I Would Do Next	4

1 Goal and Scope

The project asks whether a minimal agent can specialize itself for a class of tasks. I instantiate the idea in Python bug repair on BugsInPy [1]. The agent starts as a single-call direct bug fixer and searches over a bounded configuration space (prompt constraints, reasoning strategy, tool availability, examples, temperature). The scope is deliberately narrower than SWE-bench [2]: repairing localized Python bugs given a failing test output and candidate buggy line ranges.

2 Benchmark and Splits

The first iteration used a synthetic benchmark of 90 tasks built via mutation testing on Exercism Python exercises (single-point bugs: off-by-one, wrong operator, missing guard, wrong variable). After running the vanilla baseline, `gpt-4o-mini` solved $\sim 80\%$ with a single direct call. This left no measurable room for the stem-agent search to improve. A useful benchmark requires the base model to sit in a middle range, not near 0% and not near 100%. I discarded the synthetic set and switched to BugsInPy [1], where the same model solves roughly 25–30% under oracle fault-localization.

The benchmark is a frozen subset of 26 real BugsInPy bugs from `youtube-dl`, stored in `benchmark/subset.json`. The prompt uses a **fault-localization oracle**: snippets around the reference patch’s line ranges are shown, but the fixed code is never shown. This isolates patch generation from bug localization; quantifying the oracle’s advantage would require a parallel full-file evaluation, left for future work.

This is a pilot subset, not the benchmark I would use for a final evaluation. The intended workflow was to collect a larger set of BugsInPy bugs, validate each bug locally to make sure the checkout, dependencies, and test command were not broken by environment drift, and then execute the agent comparison on the validated set. The time budget was enough to build and run the pipeline on this small subset, but not to expand and validate the benchmark properly.

The split is fixed in `benchmark/splits.json`:

Split	Size	Role
Vital signs	4	Regression-safety invariant
Calibration	6	Sensor analysis only
Dev	8	Champion/challenger selection
Test	8	Final held-out evaluation

Statistical power. With $n=8$ binary outcomes, a one-sided exact binomial test at $\alpha=0.05$ against $p_0=0.25$ requires $\geq 5/8$ successes to reject H_0 ; only a ≥ 37.5 pp improvement is detectable. The numbers below should therefore be read as a pipeline check and qualitative failure analysis, not as evidence about the real task distribution.

3 Stem-Agent Mechanism

The agent genome is the `AgentConfig` dataclass: system prompt, few-shot examples, output format, enabled tools, approach (`direct/cot/react/plan_execute`), max iterations, and temperature. The validator rejects malformed configs.

The **mutator** implements 15 operators in two tiers: *surface-level* (12): prompt rephrasing, constraint injection, category specialization, few-shot management, tool toggle, approach/format cycling, temperature/iteration adjustment; *structural* (3): self-verification (mentally re-run the test on the fix), reflect-and-revise (second-pass self-critique), error-analysis protocol (structured root-cause classification). The space is deliberately bounded to serializable config changes. Every point is

reproducible and inspectable, answering a focused question: *can sensor-guided search beat random search within this space?*

The **differentiation loop** samples 1–2 mutations per iteration, evaluates the challenger on dev + vital signs, and promotes only if dev pass@1 improves by `>min_improvement`. The **safeguard** rejects challengers that regress vital signs below a threshold (0.75). The **sensor** runs once on calibration results, extracting failure patterns and suggesting mutation directions. The sensor uses reference fixes to analyze failure patterns in calibration tasks but never exposes them to the inner agent (`reference_fixes_used=true` in `SensorReport`, scoped to calibration only). Anti-leakage: a unit test verifies that test-suite source code never appears in the agent prompt.

Note: In the 9-iteration run only 7 of 15 operators were sampled (specialize_for_category, add_constraint, switch_approach, adjust_temperature, remove_constraint, rephrase_system, enable_tool + add_exemplar). The three structural operators were not sampled; with more iterations or seeds, coverage would increase.

4 Current Evidence

Model selection and baseline

`gpt-5.4-mini-2026-03-17` was selected over `gpt-5.4-nano` (7/25 vs. 6/25 on the full benchmark). On the 8-task test split, the vanilla direct baseline solves 2/8.

Test-split results

Agent	Solved	pass@1	95% CI
<code>vanilla_direct</code>	2/8	25.0%	[0.0, 62.5]
<code>stem_agent</code>	2/8	25.0%	[0.0, 62.5]
<code>hand_tuned</code>	1/8	12.5%	[0.0, 37.5]
<code>no_safeguard</code>	1/8	12.5%	[0.0, 37.5]
<code>no_sensor</code>	1/8	12.5%	[0.0, 37.5]
<code>random_search</code>	1/8	12.5%	[0.0, 37.5]

All single-call agents (direct, CoT) cost $\sim \$0.003$. `hand_tuned` uses ReAct with tool calls, requiring $\sim 100\times$ more tokens for the same tasks with no accuracy gain in this pilot run. That is a cost warning for this setup; it does not settle multi-step tool use in general.

McNemar $p=1.0$ for all pairs. The stem agent matches vanilla at 2/8 but solves a *different* pair (`youtube-dl:10` instead of `:25`), suggesting that the CoT mutation changed the failure-mode profile rather than aggregate accuracy. On a larger benchmark this shift might translate to an improvement, or confirm that the two strategies trade bugs rather than add them.

Dev-set differentiation

Variant	Dev score	Iters	Accepted	Converged
<code>stem_agent</code>	0.250	9	1	yes
<code>no_safeguard</code>	0.250	7	1	yes
<code>random_search</code>	0.375	8	2	yes
<code>no_sensor</code>	0.125	10	0	no
<code>vanilla_direct</code>	0.125	–	–	–

Two run-level observations (both from a **single seed**; see caveat below): (1) Random search achieved the highest dev score but dropped on test, a pattern compatible with chasing noise on

the tiny dev split. (2) Without sensor guidance, zero mutations were accepted; within this seed, the search made no progress. Both `stem_agent` and `no_safeguard` independently found `switch_approach` (`direct`→`CoT`).

Ablation matrix

Variant	Removes	Observed effect
<code>stem_agent</code>	(full system)	Found CoT, dev 0.250
<code>no_sensor</code>	Sensor	No accepted mutations
<code>no_safeguard</code>	Safeguard	Same as full system
<code>random_search</code>	Both	Higher dev, lower test

In this run, the sensor was the component associated with an accepted mutation. The safeguard contributed less because CoT happened to preserve vital signs; a larger validated benchmark would be needed to say whether that pattern holds.

Single-seed caveat. All runs use `seed=42` with 10 iterations. The difference between 0 and 1 accepted mutations could be sampling noise. Running 3–5 seeds would show whether the same pattern repeats; this was not feasible in the one-day budget.

5 What Surprised Me

Pipeline cost. I expected the benchmark pipeline to take less time. Instead, validating BugsInPy tasks, handling local environment issues, and making evaluation reproducible took most of the work. This is why the validated subset stayed small.

Small-model brittleness. I expected `gpt-5.4-mini` to perform better as a direct fixer on localized bugs. In the pilot, it solved only a small fraction of tasks, even with fault-localized snippets. This matters for the project because the whole setup is about whether configuration and prompting can make a small agent more reliable.

6 Limitations and Failure Modes

1. **No measurable improvement.** `stem_agent` = vanilla at 2/8. The search converges to CoT, which is the natural `vanilla_cot` baseline. The most informative comparison, `vanilla_cot` on the test split, was not run. The champion’s `reasoning_instruction` is effectively identical to `vanilla_cot`’s (same default fallback); only the system prompt differs (“expert ... Return only the corrected code”). Whether that prompt wording matters requires this comparison.
2. **Small validated subset.** The 26 validated bugs leave only 8 test tasks, making any difference below ~ 37.5 pp undetectable.
3. **Single seed.** All ablation observations rest on one run. The sensor’s effect is only a run-level observation.
4. **Fault-localization oracle.** The agent sees candidate line ranges, making the task easier than real-world debugging.
5. **Bounded search space.** The 15 operators modify prompts and config only; no sub-agent composition, tool generation, or memory design.

7 What I Would Do Next

- The main next step is to expand the subset to 60+ validated bugs across multiple BugsInPy projects. I would first collect more bugs, validate each one locally to confirm that the buggy

and fixed checkouts and tests still run, discard broken tasks, and only then run the agent comparison on the validated set.

- After the benchmark is expanded, run 3–5 seeds to measure variance and check whether the sensor pattern repeats.
- Run `vanilla_cot` on the test split and compare directly.
- Evaluate without the fault-localization oracle to quantify its advantage.

References

- [1] Ratnadira Widyasari et al. BugsInPy: A database of existing bugs in Python programs to enable controlled testing and debugging studies. In *Proceedings of ESEC/FSE*, 2020.
- [2] Carlos E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *Proceedings of ICLR*, 2024.